

Lecture 14: Natural Language Generation (Cont.)

Instructor: Stephanie Schoch

CSCI 680: Natural Language Processing
Department of Computer Science, College of William & Mary
October 7, 2026



Lecture Outline

- Recap
- Decoding Strategies
 - Maximization - Finishing up Beam Search
 - Sampling

Recap

Language Generation: Inference

- At inference, our decoding algorithm defines a function to select a token from the output distribution:

$$\hat{y}_t = g(P(y_t \mid y_{<t}))$$


Decoding Algorithm

- The “obvious” decoding algorithm is to greedily choose the highest probability next token according to the model at each time step:

$$g = \arg \max$$

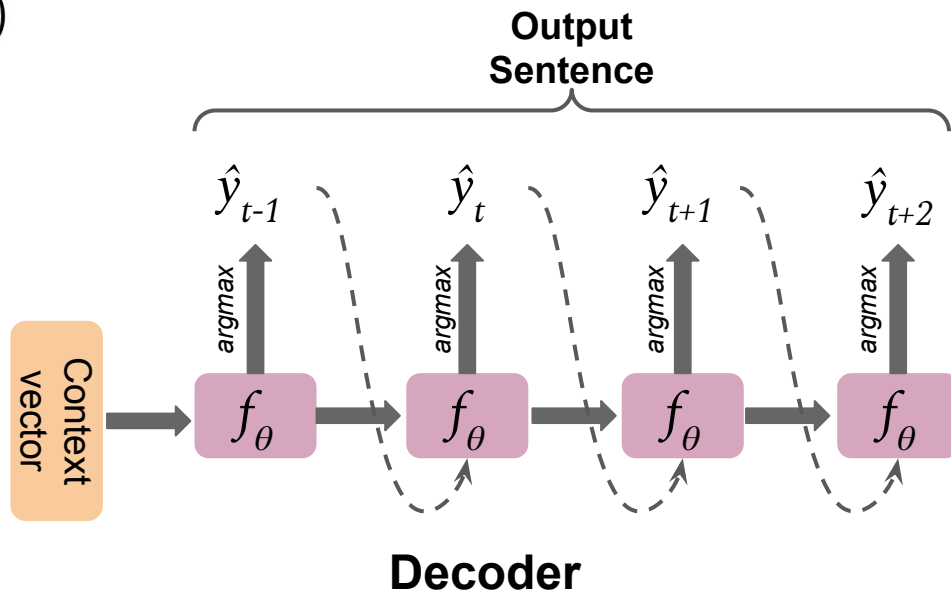
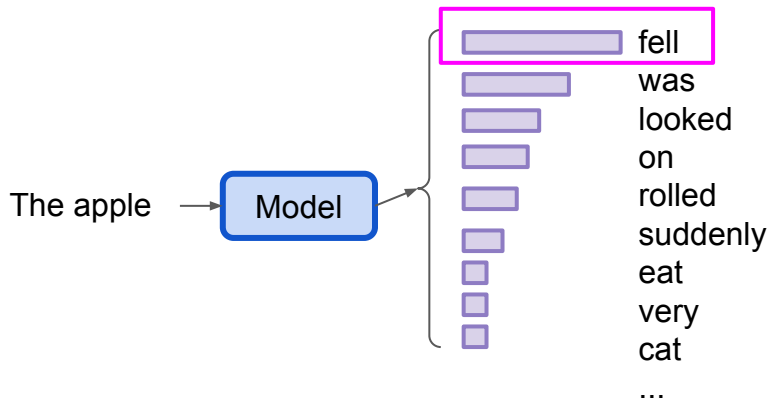
$$\hat{y}_t = \arg \max_{w \in \mathcal{V}} P(y_t = w \mid y_{<t})$$

Greedy Decoding: Argmax Decoding

- **Decoding function $g(.)$** = select highest probability token using argmax of probability distribution (most probable token at each time step).

$$\hat{y}_t = \arg \max_{w \in \mathcal{V}} P(y_t = w \mid y_{<t})$$

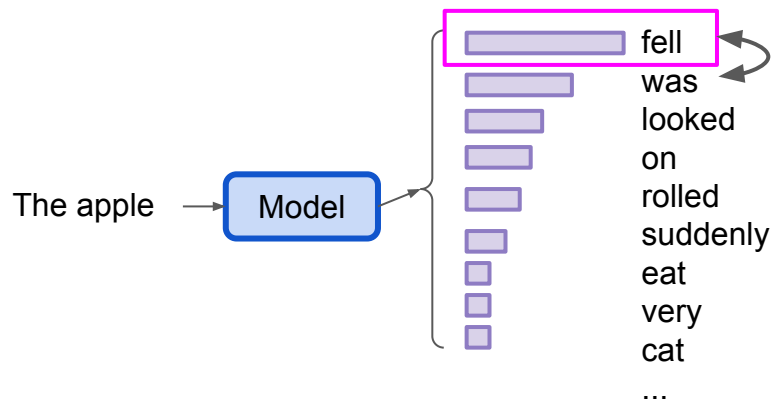
Target: The apple fell far from the tree.



Problems: Argmax Decoding

- **No backtracking. No undoing decisions.**
 - If you make a mistake, it will only propagate forward.

Target: The apple fell far from the tree.

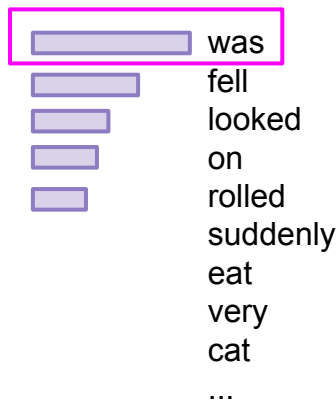


Rarely used in practice: output is often poor (ungrammatical, unnatural, nonsensical, repetitive, etc.)

Generated: The apple **was**

BUT

Wanted: The apple **fell**



How to fix this?

- Need a lookahead strategy / longer-term planning

Beam Search Decoding

- Core idea: On each step of decoder, keep track of the k most probable partial sequences (which we call hypotheses)
 - k is the beam size (in practice around 5 to 10, in NMT)
- A hypothesis has a score which is its log probability:

$$\text{score}(y_{1:t}) = \log p_{\theta}(y_{1:t} \mid x) = \sum_{i=1}^t \log p_{\theta}(y_i \mid y_{<i}, x)$$

- Scores are all negative, and higher score is better
 - We search for high-scoring hypotheses, tracking top k on each step
- Beam search is not guaranteed to find optimal solution
- But much more efficient than exhaustive search!

Beam Search Decoding: Stopping Criterion

- Greedy Decoding is done until the model produces an $\langle /s \rangle$ token
 - For e.g. $\langle s \rangle$ the apple fell far $\langle /s \rangle$
- In Beam Search Decoding, different hypotheses may produce $\langle /s \rangle$ tokens at different time steps
 - When a hypothesis produces $\langle /s \rangle$, that hypothesis is complete
 - Place it aside and continue exploring other hypotheses via beam search
- Usually we continue beam search until:
 - We reach time step T (where T is some predefined cutoff), or
 - We have at least completed n hypotheses (where n is a predefined cutoff)

Beam Search Decoding

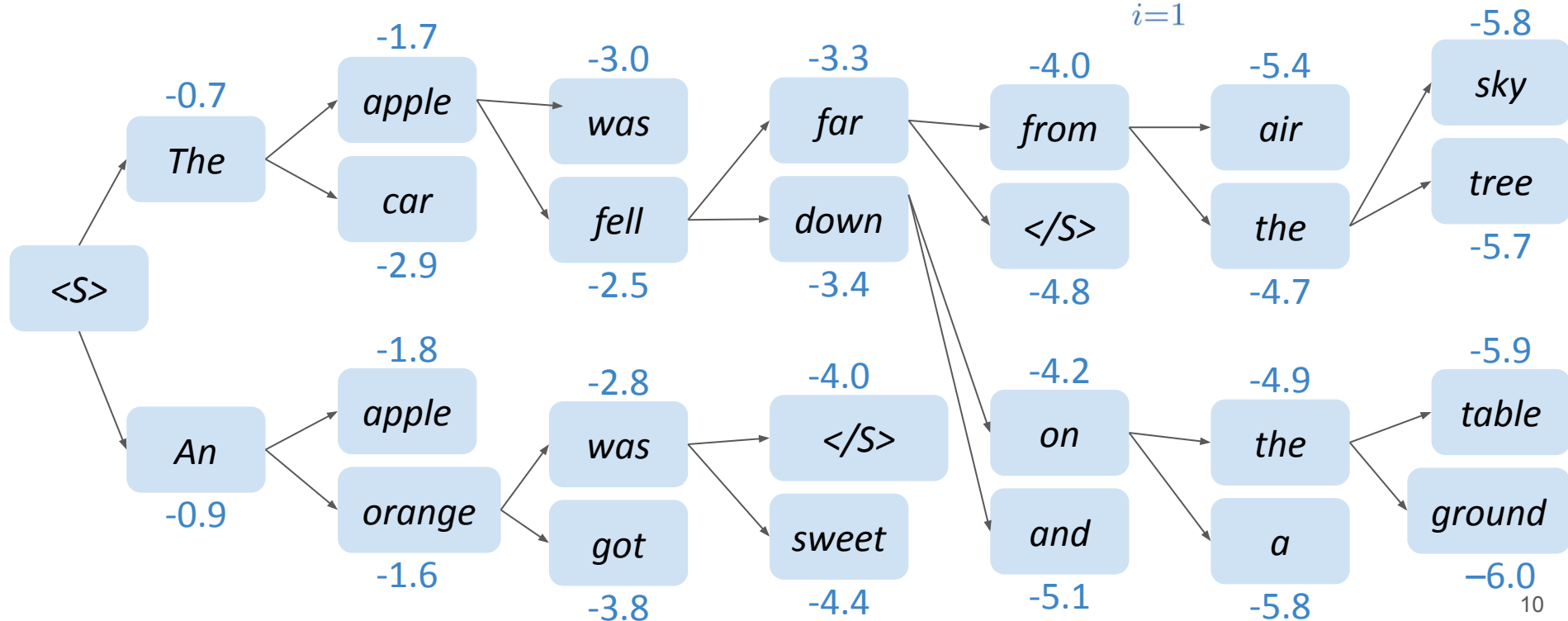
- We have our list of completed hypotheses. Now how to select top one?
- Each hypothesis $y_1, \dots, y_t$ on our list has a score

$$\text{score}(y_{1:t}) = \log p_{\theta}(y_{1:t} \mid x) = \sum_{i=1}^t \log p_{\theta}(y_i \mid y_{<i}, x)$$

Beam Search Decoding: Example

Beam size: $k = 2$

Blue numbers: $\text{score}(y_{1:t}) = \sum_{i=1}^t \log p_{\theta}(y_i \mid y_{<i}, x)$



Maximization Based Decoding

- Greedy or beam search
- Beam search can be more effective with larger beams
- **But:** generation can be bland or repetitive (i.e. degenerate)

Context: In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

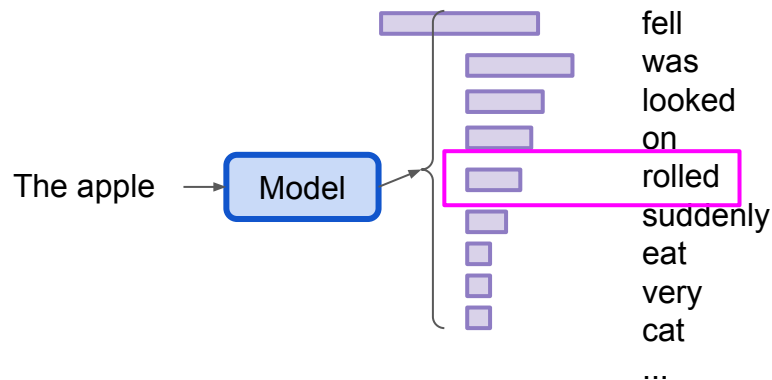
Beam Search, $b=32$:

"The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the Universidad Nacional Autónoma de México (UNAM) and the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de ..."

Decoding: Sampling

Pure Sampling

- **Decoding function $g(.)$** = sample a token from the probability distribut $P_t(w) := p_\theta(y_t = w \mid \hat{y}_{<t}, x) = \frac{\exp(S_w)}{\sum_{v \in \mathcal{V}} \exp(S_v)}$
 - Sample directly from P_t
- Still has access to the entire vocabulary
- But if the model distributions are of low quality, generations will be of low quality as well
- Often results in ill-formed generations
 - No guarantee of fluency



Beam Search vs. Pure Sampling

Context: In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

Beam Search, $b=32$:

"The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the Universidad Nacional Autónoma de México (UNAM) and the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de ..."

Pure Sampling:

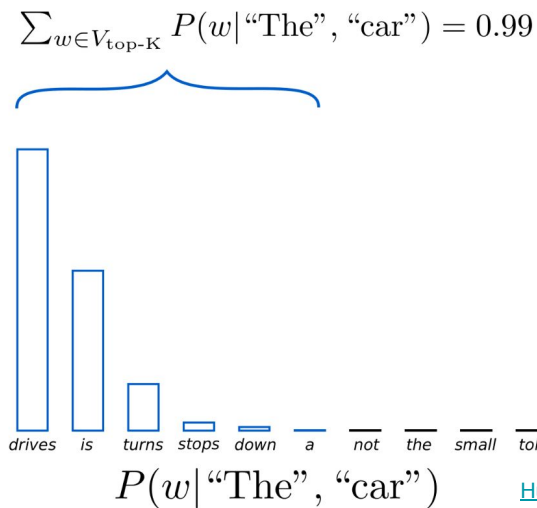
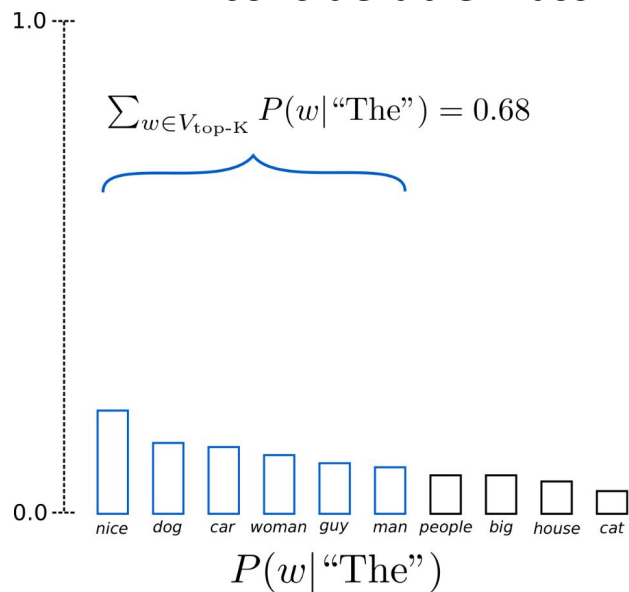
They were cattle called **Bolivian Cavalleros**; they live in a remote desert **uninterrupted by town**, and they speak **huge, beautiful, paradisiacal Bolivian linguistic thing**. They say, 'Lunch, marge.' They don't tell what the lunch is," director Professor Chuperas Omwell told Sky News. "**They've only been talking to scientists, like we're being interviewed by TV reporters. We don't even stick around to be interviewed by TV reporters. Maybe that's how they figured out that they're cosplaying as the Bolivian Cavalleros.**"

Holtzman et al., 2020

Issue: *Too much randomness!*

Top-K Sampling

- Problem: Pure sampling makes every token in the vocabulary an option
 - Even if most of the probability mass in the distribution is over a limited set of options, the tail of the distribution could be very long and in aggregate have considerable mass



Solution:

Top-K sampling

- Only sample from the top K tokens in the distribution

Top-K Sampling

- Rather than sample from entire distribution P_t , sample only from top- K most probable tokens.
 - Increase K yields more diverse, but risky outputs
 - Decrease K yields more safe but generic outputs
- $K = 1$ (argmax)
- $K = V$ (pure sampling)

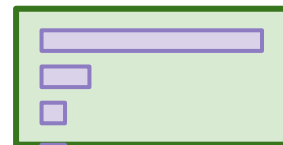
$K=3$



$t = 1$



$t = 2$

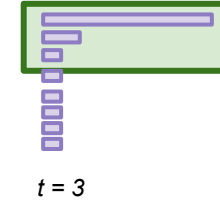
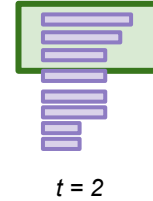
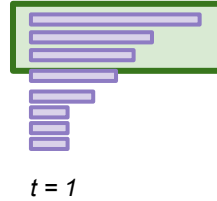


$t = 3$

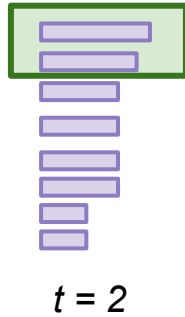
Problems: Top-K Sampling

- How to select K ?

$K=3$

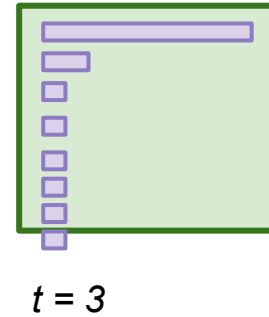


$K=2$



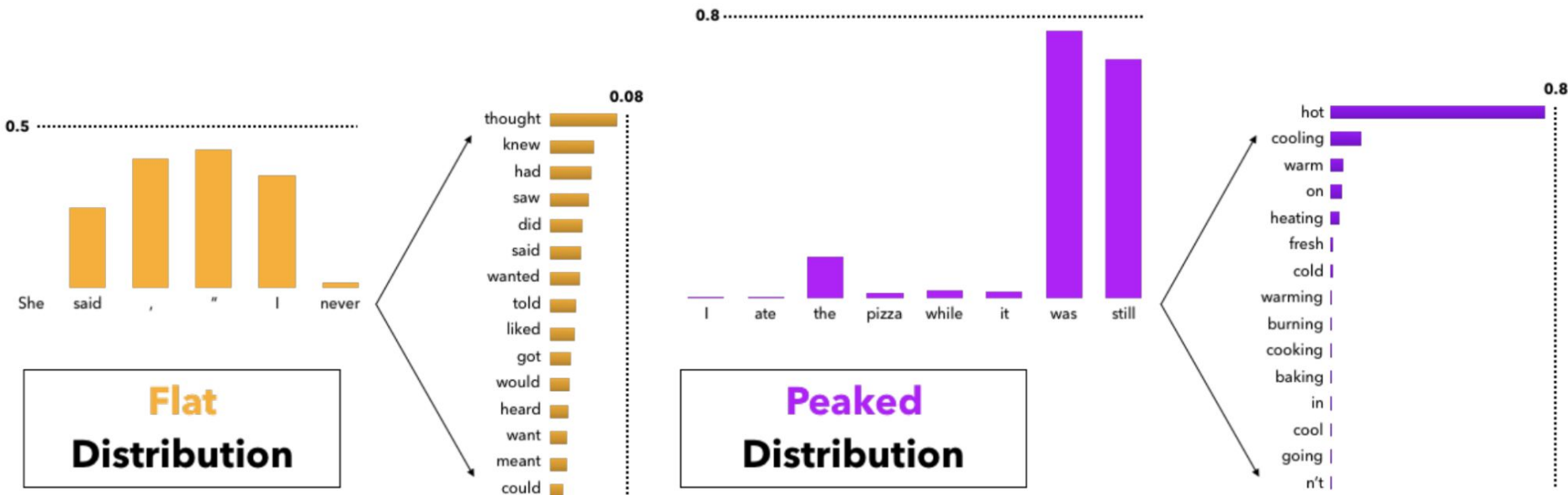
K cutoff too quickly: may not produce probable yet diverse outputs

$K=6$



K cutoff too slowly: may consider irrelevant/invalid tokens and produce nonsense

Concrete Example With Fixed K



[Holtzman et al., 2020](#)

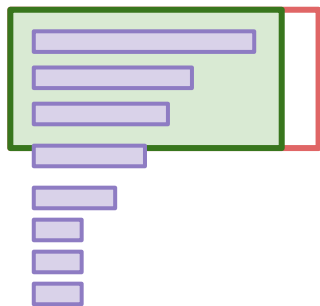
- **Problem:** The probability distributions we sample from are dynamic
 - When the distribution P_t is flatter, a limited K removes many viable options
 - When the distribution P_t is peakier, a high K allows for too many options to have a chance of being selected

Top- p Sampling

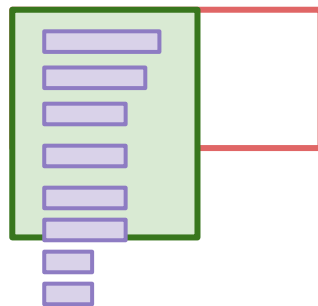
- Solution: sample from where probability mass is concentrated: **nucleus/top- P sampling**
- Sample from all tokens in the top P cumulative probability mass (i.e., where mass is concentrated)
 - Varies K depending on the uniformity of P_t
 - Nucleus is dynamically changing: add probability of each word from softmax until desired probability density of nucleus

$P^*=0.75$

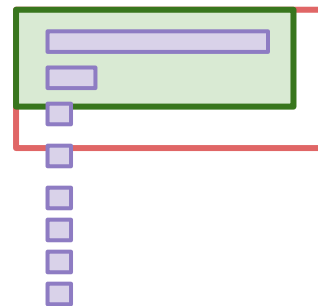
$K=3$



$t = 1$



$t = 2$



$t = 3$

Softmax Temperature Scaling

- We can add a temperature parameter to the softmax function to rebalance P_t

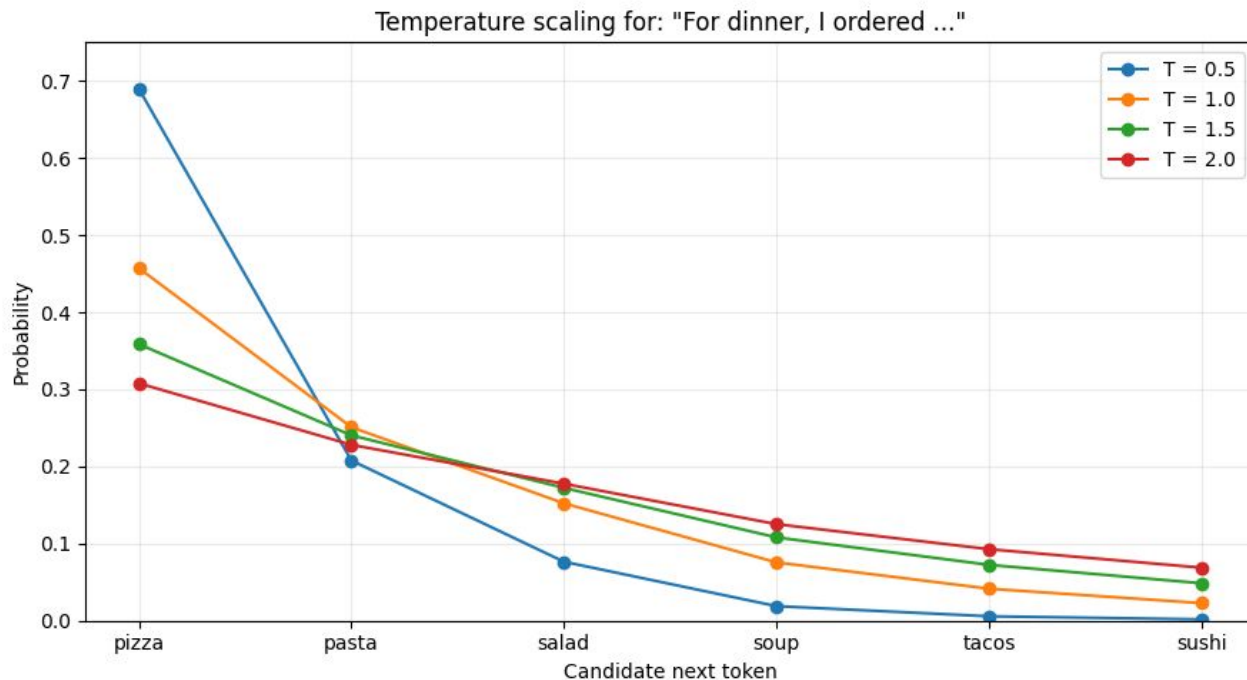
$$p_{\theta}(y_t = w) = \frac{\exp(S_w)}{\sum_{v \in \mathcal{V}} \exp(S_v)} \quad \longrightarrow \quad p_{\theta}(y_t = w) = \frac{\exp(S_w/\tau)}{\sum_{v \in \mathcal{V}} \exp(S_v/\tau)}$$

- **Raise** the temperature τ ($\tau > 1$): flatter, more uniform distribution
 - More diverse: probability spread over vocabulary
 - Potentially beyond random sampling
- **Lower** the temperature τ ($\tau < 1$): spikier, more “peaky” distribution
 - Less diverse: probability concentrated on top words
 - Closer to argmax

Temperature Scaling: Example

$$p_{\theta}(y_t = w) = \frac{\exp(S_w/\tau)}{\sum_{v \in \mathcal{V}} \exp(S_v/\tau)}$$

Token	S_w
pizza	2.2
pasta	1.6
salad	1.1
soup	0.4
tacos	-0.2
sushi	-0.8



TODOs:

- **Homework 2 Due: 10/23**
- **Quiz 2: 10/14**

Next Class: Transformers (I)